

● AULA 2 · TOKENIZAÇÃO · ABERTURA

Ninguém mudou o modelo. A conta subiu um terço.

Dois times, o mesmo produto, o mesmo modelo, o mesmo prompt de sistema — um atende em inglês, o outro em português. Fim do mês: a fatura do segundo é visivelmente maior, e a janela de contexto dele estoura primeiro com documentos do mesmo tamanho.

FATURA · TIME EN

tokens de entrada

1,00x

FATURA · TIME PT — MESMO CONTEÚDO

tokens de entrada

1,??x

o valor é medido na demo

PT

A tokeni za ç ã o é uma de cis ão

EN

Token ization is a decision

Alguém decidiu, antes do treino, o que conta como um token — e essa decisão está na fatura todo mês.

O CERCO

Os quatro têm o mesmo culpado

O token não é um dado da natureza. É uma decisão de engenharia congelada antes do treino — e ela não muda depois.

01**Dinheiro e janela**

O mesmo conteúdo em português custa mais tokens que em inglês.

por tu gu és

02**Palavra longa**

inconstitucionalissimamente sai em seis pedaços num tokenizador e em dois no outro.

incons titu cional ...

03**Código**

Um arquivo Python bem indentado gasta o orçamento de contexto em espaço em branco — e o assistente trunca antes do fim.

```
....if x:
.....return
```

04**A dívida da Aula 1**

Perplexidade de dois modelos só é comparável com o mesmo tokenizador. Por quê?

resposta no slide 14

Perplexidade é medida por token. Se dois modelos não concordam sobre o que é um token, eles não estão dividindo pela mesma coisa.

A lista que nunca fecha

01 Vocabulário aberto

Nome próprio, neologismo, URL, hashtag, identificador, erro de digitação.

02 Morfologia

correr

corri

correríamos

correndo

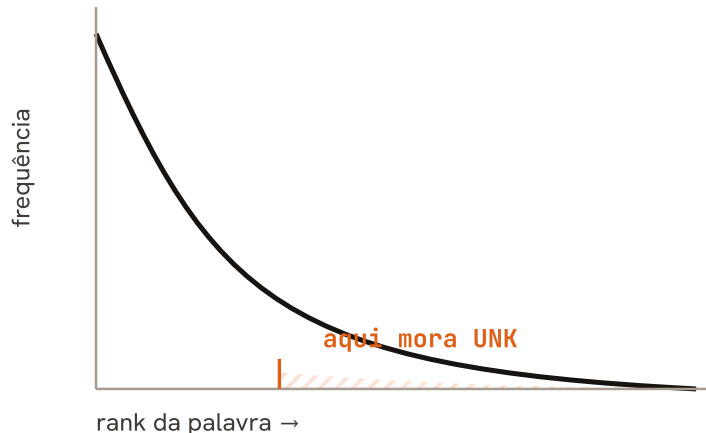
corrida

Cinco itens sem nenhuma relação declarada entre eles.

03 Zipf

Cauda longa praticamente infinita. O que cai fora vira UNK — e UNK apaga informação de forma irreversível.

“Então faz um vocabulário maior”: a cauda não termina, e cada item custa d parâmetros de embedding mais uma linha no softmax — para uma palavra com três exemplos de treino.



Palavra não serve porque a lista nunca fecha. E quando ela não fecha, o que sobra é um buraco no meio da frase que o modelo não tem como preencher.

COMPARAÇÃO · O EXTREMO DE BAIXO

Zero OOV, sequência cinco vezes mais longa

GANHA Vocabulário de dezenas de símbolos · zero OOV · qualquer string representável.

a inovação começa pelas pessoas

PAGA **Capacidade:** o modelo gasta camadas aprendendo ortografia antes de chegar à semântica.

a i n o v a ç ã o c o m e ç a p e l a s p e s s o a s

PAGA **Compute:** a atenção compara cada posição com todas as outras — o termo cresce com o **quadrado** do número de posições.

HONESTO Caractere é competitivo em correção ortográfica, transliteração e línguas sem espaço — e existe um nível abaixo, o byte.

FUNDAMENTO

O custo por camada tem dois termos: $O(n \cdot d^2)$ nas projeções e no feed-forward, $O(n^2 \cdot d)$ na atenção. Qual domina depende de n contra d . → contagem completa: **Apêndice A.5** (slide 19)

5

27

É a diferença entre ler uma frase e ler a mesma frase soletrada. A informação é a mesma, o esforço não.

CONCEITO · O COMPROMISSO

Vocabulário fechado, cobertura aberta

- Palavra frequente → um token inteiro
- Palavra rara → decomposta em pedaços já conhecidos
- $|V|$ fixo (30k-128k) e **zero OOV** — no pior caso desce até os símbolos base



Quem decide quais pedaços entram no vocabulário? É a pergunta do próximo slide.

CONCEITO · O FUNDAMENTO NOMEADO

O tokenizador treinado não é um dicionário. É uma lista ordenada.

O ALGORITMO, EM QUATRO OPERAÇÕES

```
vocabulário inicial = símbolos base
contagem           = pares adjacentes,
                    ponderados por frequência
fusão              = o par mais frequente
                    vira um símbolo novo
repetição          = até atingir o |V| alvo
```

O RESULTADO SOBRE `casa`×5 · `casas`×3 · `casinha`×2 · `casarão`×1

```
a+S → as · c+as → cas · cas+a → casa
casa   → casa
casas  → cas | as
casinha → cas | i | n | h | a
```

Três fusões, e a palavra mais frequente virou um token único — enquanto casinha continua em cinco pedaços. É exatamente o comportamento pedido no slide anterior.

```
treino = construir a lista, uma vez · codificação = aplicar a lista, sempre
```

FUNDAMENTO Execução resolvida merge a merge, com contagens, desempate, codificação de palavra nova e custo → **Apêndice A.1** (slide 15)

COMPARAÇÃO

Frequência, verossimilhança, poda — e SentencePiece não é nada disso

BPE

Funde o par **mais frequente**. Constrói de baixo para cima.

saída sem marcador de fronteira obrigatório

WordPiece (BERT)

Funde o par que mais aumenta a **verossimilhança** — prefere pares que se atraem, não os que só são frequentes.

$$\text{score} = \text{freq}(AB) / (\text{freq}(A) \cdot \text{freq}(B))$$

cas + ##inha

Unigram (Kudo)

Começa grande e **poda** os itens cuja remoção custa menos verossimilhança. De cima para baixo.

segmentação probabilística → dá para amostrar

SentencePiece

É **implementação**, não algoritmo: treina no texto cru, sem pré-tokenizar por espaço, representa o espaço como `_`, detokeniza exato — e roda **BPE e Unigram**.

FUNDAMENTO O score do WordPiece é o ganho aproximado de log-verossimilhança da fusão → **A.2** (slide 16)

FUNDAMENTO O modelo do Unigram, a busca por programação dinâmica e o critério de poda → **A.3** (slide 17)

BPE cola de baixo para cima, Unigram poda de cima para baixo — SentencePiece roda os dois.

CONCEITO

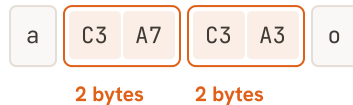
256 bytes: nada mais é OOV

- Símbolos base = os **256 bytes**, não caracteres Unicode → emoji, ideograma, byte corrompido: tudo representável.
- O vocabulário do GPT-2 **não tem** token UNK, porque não precisa.
- Preço em UTF-8: ASCII = 1 byte · ç ã õ = **2 bytes cada** → çãõ entra como 5 símbolos base antes de qualquer merge.
- Se os merges vieram de corpus inglês, não existe merge para çãõ — a sequência de bytes fica exposta.

O problema não é o byte-level: um BPE byte-level treinado em português tem merge para çãõ, mente, ções.

O GPT-2 não tem UNK porque não precisa: tudo é byte. O preço é que cada acento nosso custa dois bytes antes de qualquer merge.

ação



action



o corpus dos merges é a variável, não a técnica

DEMONSTRAÇÃO AO VIVO · 12 MIN

Um texto, quatro tokenizadores

aqui entra o terminal

gpt2	BPE byte-level
bert-base-uncased	WordPiece (EN)
neuralmind/bert-base-	
portuguese-cased	WordPiece (PT)
xlm-roberta-base	Unigram / SP

- 1 · tokens literais PT e EN, peças visíveis
- 2 · fertilidade e a razão PT/EN, com desvio
- 3 · casos difíceis: palavra longa, número, código, emoji
- 4 · mini-BPE do zero → a mesma lista do slide 6

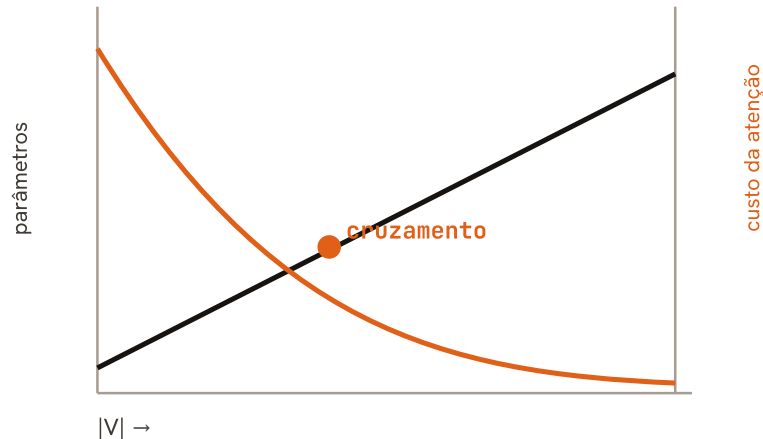
Eu não vou dizer quanto o português custa mais caro — eu vou medir na tela, e a gente anota o número que aparecer.

Trinta por cento menos tokens, quarenta e nove por cento menos atenção

- $|V|$ **grande** → sequência curta, paga em parâmetro: embedding $|V| \cdot d$ e uma linha de softmax por item.
- $|V|$ **pequeno** → sequência longa, paga na atenção, cujo termo cresce com n^2 .
- O ganho quadrático só vale onde a atenção domina, isto é, quando n é grande em relação a d . Em contexto curto, quem domina é o feed-forward — e o ganho é **linear**.
- O erro clássico: o custo é **linear** no preço de API e **quadrático** no compute da atenção — duas curvas diferentes.

$$0,7^2 = 0,49$$

a razão n/d é o interruptor entre os dois regimes



FUNDAMENTO Encurtar a sequência por um fator p multiplica o termo de projeções por p e o de atenção por p^2 → **Apêndice A.5** (slide 19)

Fora do regime da atenção o ganho é linear — e continua sendo a alavanca mais barata que existe.

COMPARAÇÃO

Números que somam errado e código que estoura o contexto

Números

2024

vs

20

27

Aritmética sobre pedaços arbitrários. **Mitigação:** forçar dígito isolado ou grupos de três.

Código

```
....for i in xs:  
.....total += i
```

Runs de indentação viram tokens; o sintoma é o assistente **truncando o arquivo antes do fim**. **Mitigação:** tokens dedicados a 4, 8, 16 espaços.

Línguas com poucos recursos

ag

ro

ne

g

óc

io

Corpus dos merges dominado pelo inglês → pedaços de 1-2 caracteres → fertilidade alta. A mesma mecânica do português, mais severa.

O erro: atribuir ao modelo o déficit que é do tokenizador. É diagnosticável **antes** de treinar e não se corrige com mais dados.

Quando o modelo erra uma soma, parte da culpa pode ser de quem cortou o número em dois.

DADOS

A janela é medida em tokens, não em palavras

Fertilidade = tokens por palavra. Razão PT/EN sobre corpus paralelo: acima de 1 — o valor é o medido na demo, anotado no quadro.

TOKENIZADOR	PT	EN	RAZÃO
gpt2			
bert-base-uncased			
bert-base-portuguese			
xlm-roberta-base			
preencher com o que a demo mediu			

Três causas somadas: corpus dos merges majoritariamente inglês · acentos = 2 bytes em byte-level · palavras mais longas e mais flexionadas.

DINHEIRO

Cobrança por token na entrada e na saída → o mesmo conteúdo sai mais caro, toda chamada. É a fatura do slide 1.

JANELA

Cabe menos conteúdo: o documento em PT consome mais janela que a tradução dele.

Não é “escrever tudo em inglês”: tem custo de manutenção e de fidelidade ao domínio. A alavanca é escolher modelo/tokenizador da língua e **medir antes de decidir**.

FUNDAMENTO A média das razões par a par **não** é a razão dos totais — a segunda é ponderada por comprimento → **Apêndice A.4** (slide 18)

O mesmo documento em português ocupa mais janela do que a tradução dele — é a mesma bagagem numa mala

EXERCÍCIO EM DUPLA

Qual é a causa, e que conta confirma

08:00

01

Um time atende em português e em inglês, mesmo produto e mesmo modelo. A fatura do lado português é visivelmente maior — e ninguém mudou nada no código.

causa

conta que confirma

02

Uma conversa de suporte com 20 turnos custou **muito** mais que 20 vezes um turno. O time orçou por “custo de um turno x número de turnos” e errou por mais de uma ordem de grandeza.

causa

conta que confirma

03

Um assistente com janela de 8 mil tokens funciona bem em texto corrido e **trunca** arquivos de código pela metade — mesmo com arquivos que têm menos palavras que os documentos.

causa

conta que confirma

HIPÓTESE DO EXERCÍCIO US\$ 0,50 por milhão de tokens de entrada — não é preço de provedor real

Nenhuma das três se resolve decorando fórmula. As três se resolvem sabendo que conta fazer.

Perplexidade é por token

$$\text{PPL} = \exp\left(-\left(\frac{1}{N}\right) \sum \log P(t_i \mid t_{<i})\right)$$

- Muda N: o mesmo texto tem contagens diferentes de token — **medido na demo**.
- Muda o espaço de eventos: P está definida sobre o vocabulário **daquele** tokenizador.
- Logo: não é comparação imprecisa, é comparação **sem sentido**.
- Alternativa honesta: normalizar por unidade invariante — **bits por caractere** ou por byte.
- Reescalar pela razão de fertilidade não resolve: a relação é uma **potência**, não um fator.

A PONTE PARA A AULA 3

A	inova	ção	começa
4021	318	257	?

ÍNDICE DO APÊNDICE

A.1 BPE resolvido · A.2 WordPiece · A.3 Unigram
A.4 fertilidade · A.5 custo $O(n^2)$ · A.6 perplexidade

O token é uma decisão de engenharia congelada antes do treino: define quanto vocês pagam, quanto texto cabe e se a métrica de dois modelos pode ser comparada.

Aula 3 — Embeddings: o ID 4021 não é parecido com o 4020. O que faz um inteiro significar algo?

A.1 O algoritmo do BPE, resolvido merge a merge

invocado no slide 6

NOTAÇÃO E ALGORITMO

Σ_0 símbolos base · $W = \{(w, f(w))\}$
 $s(w)$ sequência atual de w
 $c(A,B) = \sum f(w) \cdot \text{\#adjacências}$
 $M = [m_1, \dots, m_k]$ lista ordenada
 $|V| = |\Sigma_0| + k$

```
1  $s(w) \leftarrow w$  partida em  $\Sigma_0$ 
2  $M \leftarrow []$ 
3 repita k vezes:
4   calcule  $c(A,B)$  de todo par
5    $(A^*, B^*) \leftarrow \text{argmax } c(A,B)$ 
6    $M \leftarrow M + [(A^*, B^*)]$ 
7   substitua  $A^*B^*$  em toda  $s(w)$ 
```

Merges não atravessam fronteira de palavra. Trocar o corpus troca a lista — é essa a variável do português caro do slide 12.

EXECUÇÃO · CASA×5 CASAS×3 CASINHA×2 CASARÃO×1

```
R1 (c,a) 11 (a,s) 14 ← (s,a) 9
   (s,i) 2 (i,n) 2 (n,h) 2
merge 1: a+s → as

R2 (c,as) 11 ← (as,a) 6 (as,as) 3
merge 2: c+as → cas

R3 (cas,a) 6 ← (cas,as) 3 (cas,i) 2
merge 3: cas+a → casa

R4 (cas,as) 3 ←
merge 4: cas+as → casas
```

$M = [(a,s), (c,as), (cas,a), (cas,as)]$

R5 · o empate. $(cas,i)=(i,n)=(n,h)=(h,a)=2$: empate quádruplo, e argmax é ambíguo. Cada implementação desempata por critério próprio — por isso duas implementações corretas produzem listas diferentes, e o tokenizador viaja junto com o modelo como arquivo.

CODIFICAR PALAVRA NOVA · CASINHAS

```
c a s i n h a s
m1 (a,s) → c a s i n h a s
m2 (c,as) → c a s i n h a s
m3 (cas,a) → sem efeito
m4 (cas,as) → sem efeito

cas | i | n | h | as = 5 tokens
```

casas custa 1 token e **casinhas** custa 5, mesma raiz, mesmo tokenizador. Não há inteligência: uma estava frequente no corpus, a outra não.

CASOS-LIMITE E CUSTO

Sem OOV: no pior caso desce a Σ_0 · k grande demais memoriza palavras do treino e incha $|V|$ · Σ_0 = bytes elimina o inexprimível ao preço de 2 símbolos por acento · treino ingênuo $O(k \cdot N)$, codificação $O(k \cdot |w|)$ — na prática $O(|w| \log |w|)$ com fila de prioridade.

Sennrich, Haddow & Birch — arXiv 1508.07909

A.2 WordPiece: de verossimilhança a $\text{freq}(\text{AB})/(\text{freq}(\text{A})\cdot\text{freq}(\text{B}))$

invocado no slide 7

Modelo unigrama com $p(x) = c(x)/N$. Premissas: corpus como saco de tokens independentes; efeito do merge sobre os demais tokens desprezado.

$$L(V) = \sum c(x) \cdot \log(c(x)/N)$$

$$\begin{aligned}\Delta L &\approx c(\text{AB}) \cdot \log[p(\text{AB}) / (p(\text{A}) \cdot p(\text{B}))] \\ &= c(\text{AB}) \cdot \log[c(\text{AB}) \cdot N / (c(\text{A}) \cdot c(\text{B}))]\end{aligned}$$

$$\text{score}(\text{A}, \text{B}) = c(\text{AB}) / (c(\text{A}) \cdot c(\text{B}))$$

O termo dentro do log é a **informação mútua pontual** entre A e B; o fator $c(\text{AB})$ fora é quantas vezes o ganho se realiza. Descartando N, igual para todos os pares da rodada, sobra o critério do slide 7. ■

O MARCADOR

casinha → cas + ##inha. O ## separa o espaço de eventos: inha como início e ##inha como continuação são itens distintos — detokenização sem ambiguidade.

ONDE OS DOIS ESCOLHEM DIFERENTE

par	c(AB)	c(A)	c(B)	score
(d,e)	4.000	9.000	30.000	1,48e-8
(q,u)	900	910	6.000	1,65e-7

BPE	funde (d,e)	4.000 > 900
WordPiece	funde (q,u)	11× maior

q praticamente não existe sem u em português: 99% das ocorrências. Fundir qu captura uma regularidade da língua; fundir de captura uma coincidência de frequência. O BPE otimiza compressão imediata, o WordPiece otimiza verossimilhança — objetivos diferentes, nenhum errado.

CASOS-LIMITE

$c(\text{AB})=c(\text{A})=c(\text{B}) \rightarrow$ par sempre junto, caso ideal · par frequente entre tokens frequentes
→ score pequeno, ignorado · $c(\text{AB})=1 \rightarrow$ score enganoso na cauda, daí o piso de contagem
· critério guloso: nenhum dos dois garante o vocabulário ótimo.

A.3 Unigram e SentencePiece

invocado no slide 7

MODELO E BUSCA

$$\begin{aligned}P(s) &= \prod p(x_i) \\P(w) &= \sum_{\{s \in S(w)\}} P(s) \\s^*(w) &= \operatorname{argmax} P(s)\end{aligned}$$

$$\begin{aligned}D[\emptyset] &= 0 \\D[j] &= \max\{D[i] + \log p(w_{\{i+1..j\}})\} \\&\quad 0 \leq i < j, \quad w_{\{i+1..j\}} \in V\end{aligned}$$

$|S(w)|$ cresce exponencialmente, mas s^* sai por programação dinâmica: com item de comprimento máximo L , o custo é $O(|w| \cdot L)$. É Viterbi sobre uma cadeia.

TREINO POR PODA

- 1 · com V fixo, ajustar $\{p(x)\}$ maximizando $\sum f(w) \cdot \log P(w)$ — segmentações são latentes, o ajuste é por maximização de esperança.
- 2 · com $\{p(x)\}$ fixo, medir o custo de remover cada item.
- 3 · remover os de menor `loss`, mantendo $(1-\eta)$ do vocabulário; símbolos base nunca saem.
- 4 · repetir até o $|V|$ alvo.

$$\text{loss}(x) = L(V) - L(V \setminus \{x\}) \geq 0$$

BPE pergunta “que fusão comprime mais agora?”. Unigram pergunta “de que item eu menos sinto falta?”. Direções opostas no mesmo espaço.

SUBWORD REGULARIZATION

Como $P(s|w) = P(s)/P(w)$ é distribuição, dá para amostrar em vez de tomar a melhor. Com temperatura α : $\alpha \rightarrow \infty$ recupera Viterbi, $\alpha=1$ amostra do modelo. Funciona como aumento de dados.

SENTENCEPIECE

Treina no texto cru, sem pré-tokenização · o espaço é um caractere do vocabulário, `_` · detokenização exata, byte a byte. “O modelo usa SentencePiece” **não informa** qual algoritmo roda — e é isso que decide se a saída tem `_`, `##` ou nada.

Casos-limite: item sem uso é podado na 1ª rodada · símbolo base nunca sai · η grande piora o vocabulário · amostrar na inferência faz N variar, e a perplexidade deixa de ser bem definida.

Kudo 1804.10959 · Kudo & Richardson 1808.06226

A.4 Fertilidade, e as duas médias que não são a mesma

invocado no slide 12

DEFINIÇÃO

$$\text{fertilidade}(T, x) = |T(x)| / \text{palavras}(x)$$

Premissas: “palavra separada por espaço” é convenção e precisa ser declarada · [CLS]/[SEP] inflam o numerador em 2 e distorcem textos curtos · fertilidade é propriedade do **par** (T, x), nunca do tokenizador isolado.

Não existe “a fertilidade do GPT-2”. Existe a dele em português jornalístico, em código Python, em inglês médico — e elas diferem. Reportar sem dizer o corpus é reportar meio número.

AS DUAS MÉDIAS

$$r_{\text{médio}} = (1/n) \sum t_i / u_i$$

$$r_{\text{total}} = (\sum t_i) / (\sum u_i)$$

$$= \sum (u_i / \sum u_j) \cdot p_i$$

$$\sigma_p = \text{sqrt}((1/(n-1)) \sum (p_i - r_{\text{médio}})^2)$$

r_{total} pondera cada par pelo comprimento do lado inglês; $r_{\text{médio}}$ dá o mesmo peso a um par de 3 e a um de 80 palavras. Coincidem se e somente se todos os p_i ou todos os u_i forem iguais. ■

Para **custo**, r_{total} : a fatura soma tokens, não médias de razões. Para **caracterizar** o tokenizador, $r_{\text{médio}}$ com desvio. Reportar r sem σ esconde se o pior par é 1,4 ou 3,0 — e é o pior par que estoura a janela.

PROPAGAÇÃO

$$\begin{aligned} \text{custo} &= \pi \cdot \text{fertilidade} \cdot p && \text{linear} \\ \text{páginas} &= W / (\text{fertilidade} \cdot p) && \text{inversa} \end{aligned}$$

$$\begin{aligned} r &= 1,33 \rightarrow +33\% \text{ na fatura} \\ &\rightarrow -25\% \text{ de páginas } (1 - 1/1,33) \\ W &= 8.192, p = 500, \text{ fert } 1,5 \rightarrow 10,9 \text{ páginas} \\ &\text{fert } 2,0 \rightarrow 8,2 \text{ páginas} \end{aligned}$$

“1 token $\approx$ 4 caracteres” é medição em inglês, não teorema. Em byte-level o que se consome são bytes: com 8% de caracteres acentuados são 1,08 bytes por caractere, e todo o resto herda o fator antes de qualquer merge.

Casos-limite: texto de 1 palavra em BERT $\rightarrow$ fertilidade 3,0, correta e inútil · em código, normalizar por linha ou byte · fertilidade < 1 é possível · corpus paralelo mal alinhado mede tradução, não tokenização.

A.5 Vocabulário, comprimento e o custo $O(n^2)$

invocado nos slides 4 e 10

CUSTO POR CAMADA, TERMO A TERMO

Q, K, V e saída	$O(n \cdot d^2)$
afinidade QK ^T	$O(n^2 \cdot d)$
softmax atenção	$O(n^2)$
agregação	$O(n^2 \cdot d)$
feed-forward 4x	$O(n \cdot d^2)$

$\text{custo} \approx c_1 \cdot n \cdot d^2 + c_2 \cdot n^2 \cdot d$

Depende de $|V|$: embedding $|V| \cdot d$, projeção de saída $|V| \cdot d$, softmax $n \cdot |V| \cdot d$ por passada. E cada item novo recebe menos exemplos, pela cauda de Zipf.

O INTERRUPTOR N/D

$(c_2 \cdot n^2 \cdot d) / (c_1 \cdot n \cdot d^2) = (c_2 / c_1) \cdot (n / d)$

$d = 4.096$

$n = 2.048 \rightarrow n/d = 0,5$ FFN domina

$n = 4.096 \rightarrow n/d = 1$ mesma ordem

$n = 32.768 \rightarrow n/d = 8$ atenção domina

ENCURTAR POR UM FATOR P

projeções e FFN $\rightarrow p$

atenção $\rightarrow p^2$

preço de API $\rightarrow p$

$p = 0,70 \rightarrow 30\%$ de economia

$p^2 = 0,49 \rightarrow 51\%$ no termo de atenção

MEMÓRIA, O LIMITE QUE ESTOURA PRIMEIRO

$\text{memória} \approx B \cdot h \cdot n^2 \cdot L \cdot \text{bytes}$

Dobrar n quadruplica esse termo, e é ele que decide o teto de contexto viável antes de qualquer consideração de tempo. É o alvo do FlashAttention (Aula 12) e a conta que a Aula 8 refaz para o KV cache.

POR QUE O $|V|$ ÓTIMO É ACHATADO

$n(|V|)$ é decrescente e convexa (Zipf: o k -ésimo merge cobre menos texto que o anterior), enquanto $2 \cdot |V| \cdot d$ é linear. A soma tem mínimo interior, mas raso — daí a faixa 30k–128k em vez de um valor. A decisão que importa não é o tamanho do vocabulário, é o corpus em que os merges foram aprendidos.

Casos-limite: $n=1$ só sobra projeção (inferência com cache) · $|V| \models |\Sigma_o|$ morre na memória $n^2 \cdot |V|$ enorme traz o UNK de volta · com d crescendo, a maior parte do compute não está na atenção.

A.6 Perplexidade é por token: o que é invariante e o que não é

invocado no slide 14

INVARIANTE E NÃO INVARIANTE

Com tokenização única e sem perda, $P_A(x) = P_A(T_A(x))$: $\log P(x)$ é comparável — mesma unidade, mesmo objeto, mesma pergunta.

$$\begin{aligned} H_A &= -\log P_A(x) / N_A \\ PPL_A &= \exp(H_A) \end{aligned}$$

O numerador é comparável; o **denominador não**, porque N é escolha do tokenizador. Perplexidade é média por passo, e os dois modelos não têm o mesmo número de passos para dizer a mesma coisa.

A CONVERSÃO É UMA POTÊNCIA

$$\begin{aligned} H_A \cdot N_A &= H_B \cdot N_B \\ r = N_A / N_B &\rightarrow PPL_B = PPL_A^r \end{aligned}$$

$$\begin{aligned} PPL_{car} = 4,0 &\rightarrow H = 1,386 \text{ nat/car} \\ r = 4 &\rightarrow H_{sub} = 5,545 \\ &\rightarrow PPL_{sub} = 256 \end{aligned}$$

4,0 e 256, mesmo modelo, mesmo texto. A razão de 64 veio inteira da escolha de r . Dividir a perplexidade pela razão de fertilidade corrige uma multiplicação onde a relação é exponenciação.

E corrigir a unidade não basta: prever entre 90 opções e prever entre 32.000 são tarefas diferentes. Não é imprecisão de escala, é troca de pergunta.

A NORMALIZAÇÃO COMPARÁVEL

$$\begin{aligned} BPC &= -\log_2 P(x) / N_{caracteres} \\ &= \log_2(PPL_{token}) \cdot (N_{tok} / N_{car}) \end{aligned}$$

$$\begin{aligned} \text{caractere: } \log_2(4,0) \cdot 1 &= 2,00 \\ \text{subword: } \log_2(256) / 4 &= 2,00 \quad \checkmark \\ \text{real: } \log_2(20) / 4 &= 1,08 \end{aligned}$$

Contra 2,00, o modelo de subword é substancialmente melhor — conclusão que só aparece depois da normalização. Bits por **byte** é ainda mais robusto (não depende da convenção de caractere em Unicode), mas favorece levemente línguas com muito ASCII: nenhuma normalização é neutra, e vale dizer qual se usou.

Casos-limite: tokenizador com perda (o bert-base-uncased da demo) invalida até $\log P(x)$ · tokenizador estocástico deixa N variar · conjuntos de avaliação diferentes é erro mais comum ainda · o teto trivial é $|V|$.